- The *model* generator creates a migration to in turn create the table associated with the model (unless you specify the `--skip-migration` option). As the example that follows shows, creating a model called `discount` also creates a migration called *yyyyMMddhhmmss*`_create_discounts.rb`:

```
depot> bin/rails generate model discount
    invoke  active_record
»   create    db/migrate/20250420133549_create_discounts.rb
    create    app/models/discount.rb
    invoke    test_unit
    create      test/models/discount_test.rb
    create      test/fixtures/discounts.yml
```

- You can also generate a migration on its own.

```
depot> bin/rails generate migration add_price_column
    invoke  active_record
»   create    db/migrate/20250420133814_add_price_column.rb
```

Later, starting in *Anatomy of a Migration*, we'll see what goes in the migration files. But for now, let's jump ahead a little in the workflow and see how to run migrations.

Running Migrations

Migrations are run using the `db:migrate` Rake task:

```
depot> bin/rails db:migrate
```

To see what happens next, let's dive down into the internals of Rails.

The migration code maintains a table called `schema_migrations` inside every Rails database. This table has just one column, called `version`, and it will have one row per successfully applied migration.